

《人工智能——深度学习大模型智能体》

配套实验指导书

实验作业一

使用 TRAE IDE 设计更好的神经网络

（对应教材第 2 章《神经元与神经网络》）

一、实验目的

- 1. 掌握人工神经元与多层感知器（MLP）的基本结构，理解加权和、偏置与激活函数在模型中的作用；
- 2. 掌握神经网络的完整训练流程：前向传播、损失计算、反向传播与参数更新；
- 3. 通过控制变量对比实验，理解激活函数、优化器、学习率、正则化（L2、Dropout）与 BatchNorm 对模型性能的影响；
- 4. 学会使用 TRAE IDE 的 AI 能力（Chat / Builder / 内联补全）辅助代码编写、调试与实验分析，掌握“人机协作”的深度学习实验范式；
- 5. 培养严谨的实验记录习惯与基于数据的模型改进思维。

二、实验环境与准备工作

2.1 软件环境

表 1 实验软件环境清单

类别	软件 / 库	版本要求	用途
AI 编程环境	TRAE IDE	最新版（国内版即可）	代码编写、AI 辅助建模与调试
编程语言	Python	3.10 及以上	实验主语言
深度学习框架	PyTorch	2.0 及以上	构建与训练神经网络
数据处理	scikit-learn、NumPy	最新稳定版	数据集加载与预处理
可视化	Matplotlib	最新稳定版	绘制损失曲线、准确率曲线与混淆矩阵

2.2 环境搭建步骤

（1）安装 TRAE IDE：前往 TRAE 官网（trae.com 或 trae.cn）下载对应操作系统的安装包（Windows / macOS / Linux），按向导完成安装并登录账号。

（2）创建工作目录：在磁盘上新建文件夹 lab01_neural_network，并在 TRAE IDE 中通过“文件 → 打开文件夹”打开该目录。

（3）安装依赖：在 TRAE 内置终端（终端 → 新建终端）中执行以下命令：

```
python -m venv venv
# Windows 激活虚拟环境：
venv\Scripts\activate
# macOS / Linux 激活虚拟环境：
source venv/bin/activate
pip install torch torchvision scikit-learn matplotlib numpy
```

（4）验证安装：新建 verify_env.py 并运行，确认 PyTorch 可用：

```
import torch, sklearn, matplotlib
print("PyTorch:", torch.__version__)
print("CUDA 可用:", torch.cuda.is_available()) # 无 GPU 显示 False 也可正常完成本实验
```

说明：本实验全部数据集均为离线内置数据集，无需联网下载；无独立显卡的电脑同样可以完成。

三、TRAE IDE 简介

TRAE 是一款面向 AI 编程的集成开发环境（AI-native IDE），与普通编辑器相比，它把大语言模型深度集成进了编码全流程。本实验将重点使用以下三项能力：

表 2 本实验使用到的 TRAE 核心功能

功能	调用方式	在本实验中的用途
Chat 对话助手	侧边栏 Chat 面板	解释报错信息、讲解代码原理、生成实验改进方案
Builder 智能构建	Builder 模式（#Builder）	按自然语言描述一次性生成完整实验代码框架
内联补全	编辑代码时自动触发	补全训练循环、指标计算等重复性代码
智能调试	报错处点击“修复”	快速定位并修复运行错误

使用原则（重要）：TRAE 生成的代码只是“起点”而非“终点”。本实验要求你必须读懂、能修改、能解释 AI 生成的每一行关键代码，并在实验报告中说明哪些设计是你自己的决策、依据是什么。直接提交无法解释的 AI 代码将影响评分。

四、实验原理回顾（对应教材第 2 章）

4.1 神经元与多层感知器

单个神经元完成“加权和 + 偏置 + 激活”三步运算： $y = f(W \cdot X + b)$ 。多个神经元按层组织，构成多层感知器（MLP）：输入层接收特征，隐藏层逐层提取抽象特征，输出层给出预测。只有激活函数引入非线性，网络才能表达超越线性模型的函数。

4.2 训练三步循环

训练神经网络 = 反复执行三步：① 前向传播，计算预测值；② 用损失函数（回归用均方误差 MSE，分类用交叉熵 Cross-Entropy）量化预测误差；③ 反向传播按链式法则求出各参数梯度，优化器沿负梯度方向更新参数 $w \leftarrow w - \eta \cdot \partial L / \partial w$ 。

4.3 本实验将验证的关键设计点

表 3 神经网络设计要点与预期影响

设计维度	可选项	教材对应章节	预期影响
激活函数	Sigmoid / Tanh / ReLU /	2.4 节	影响收敛速度与梯度消失风险

	GELU		
网络深度与宽度	隐藏层数 1–3 层，每层 32–256 个神经元	2.2、2.3 节	容量越大拟合能力越强，但过拟合风险上升
优化器	SGD / SGD+Momentum / Adam	2.8–2.10 节	影响收敛速度与最终精度
学习率	0.1 / 0.01 / 0.001 等	2.8 节	过大震荡发散，过小收敛缓慢
L2 正则化	weight_decay = 0 / 1e-4 / 1e-3	2.11–2.12 节	抑制过拟合，缩小训练-测试差距
Dropout	p = 0 / 0.2 / 0.5	2.12 节	随机失活，增强泛化
BatchNorm	有 / 无	2.13 节	稳定训练过程，允许更大学习率

五、实验任务与总体要求

在 sklearn 内置的手写数字数据集（load_digits，8×8 灰度图，1797 个样本，10 个类别）上，从零构建一个 MLP 分类器。首先建立一个“朴素基线（Baseline）”，然后围绕表 3 中的设计维度开展至少 6 组对照实验，最终得到一个显著优于基线的“更好的神经网络”。

总体要求：

- (1) 所有实验共用同一份数据划分（训练集 80%、测试集 20% 分层抽样）与随机种子，确保对比公平；
- (2) 每组实验只改动一个变量（控制变量法），其余超参数保持与基线一致；
- (3) 每组实验必须记录测试准确率与训练耗时，并保存损失曲线图片；
- (4) 至少 3 处代码需借助 TRAE 的 AI 能力生成或修改，并在报告中注明“AI 参与”方式；
- (5) 禁止直接调用 sklearn 的 MLPClassifier 代替手写 PyTorch 训练流程。

六、实验步骤

步骤 1 用 TRAE Builder 生成实验框架（约 15 分钟）

打开 TRAE 的 Builder 模式，在对话框输入如下提示词（可直接复制）：

请在当前目录创建 mlp_digits.py，用 PyTorch 实现一个手写数字分类实验框架，要求：

1. 加载 sklearn 的 load_digits 数据集，按 8:2 分层划分训练/测试集，随机种子 42；
注意把标签转为 torch.long 类型（Windows 上 numpy 标签是 int32，CrossEntropyLoss 会报错 expected scalar type Long）；

2. 定义一个 PyTorch 的 MLP 类，隐藏层结构、激活函数、Dropout、是否使用 BatchNorm 均可通过参数配置；

3. 训练函数中优化器类型（SGD/Adam）、学习率、weight_decay 均为可配置参数，训练循环包含：前向传播、交叉熵损失、反向传播、参数更新；

4. 每个 epoch 记录训练损失与测试准确率；

5. 训练结束后绘制损失曲线和准确率曲线，保存为 result_baseline.png；

```
画图前设置 matplotlib 中文字体:
plt.rcParams["font.sans-serif"] = ["Microsoft YaHei"]
plt.rcParams["axes.unicode_minus"] = False
6. 在终端打印最终测试准确率与训练总耗时。
```

生成后通读代码，重点检查：数据划分是否分层、标签是否为 Long 类型、损失函数是否为交叉熵、优化器与学习率是否为可配置参数。若有不理解之处，选中代码段让 Chat 助手逐行解释。

步骤 2 运行并记录基线（Baseline）实验

使用最朴素的结构运行基线：输入层 64 维 → 1 个隐藏层（32 个神经元，Sigmoid 激活）→ 输出层 10 类；优化器 SGD，学习率 0.1，训练 30 个 epoch（全批量），无任何正则化。该基线“能学但学不好”——预期测试准确率明显低于 50%，这正是后续各改进组发挥作用的空间。将结果填入表 4 的第 1 行。

步骤 3 逐项改进实验（核心环节）

参照表 4 规划的实验组，每次只改动一个设计点。建议按下表顺序进行（也可自行增删，但不得少于 6 组）：

表 4 对照实验设计与结果记录表（实验后填写）

组号	实验名称	改动内容（相对基线）	测试准确率（%）	训练耗时（s）	观察结论
0	基线 Baseline	1×32 Sigmoid + SGD(0.1)	_____	_____	_____
1	换激活函数	Sigmoid → ReLU	_____	_____	_____
2	加深网络	1 层 → 2 层 × 128 神经元	_____	_____	_____
3	换优化器	SGD → Adam(0.01)	_____	_____	_____
4	调学习率	0.1 → 0.01（或自行扫描）	_____	_____	_____
5	加 L2 正则	weight_decay = 1e-4	_____	_____	_____
6	加 Dropout	p = 0.2	_____	_____	_____
7	加 BatchNorm	每个隐藏层后插入 BatchNorm1d	_____	_____	_____
8	最优组合	综合以上有效改进	_____	_____	_____

提示：利用 TRAE Chat 快速完成改动。例如可直接提问：“请把模型中的 Sigmoid 激活函数改为 ReLU，其他保持不变，并告诉我这一改动为什么通常能加快收敛。”每组实验的图片按 result_exp1.png、result_exp2.png 命名保存。

步骤 4 失败实验分析（必做）

故意构造一次“失败”：将学习率设为 10.0 再训练一次（基线其余配置不变），观察损失曲线剧烈震荡、无法稳定下降的现象，截图保存为 result_lr_too_big.png，并用 Chat 助手辅助解释为什么学习率过大会导致不收敛。这一组不计入改进组数，但必须写入报告。

提示：若时间允许，可在 0.1、1.0、5.0、10.0、20.0 之间扫描，寻找本实验设置下的“发散临界点”。你会发现 $lr=1.0$ 时模型反而能学得不错——这并不是说教材错了，而是因为本实验数据集极小、全批量更新步数少，只有更大的学习率才会把参数“甩”出稳定区。把这个反直觉现象写进报告是加分项。

步骤 5 形成最优模型并复测

把各维度中被验证有效的改进组合成“最优组合”（第 8 组），例如：ReLU + 2 层 $\times$ 128 神经元 + BatchNorm + Adam(0.001)。固定随机种子重新运行 3 次取平均，避免偶然性。若最优组合反而不如某个单改进组，分析原因（例如容量过大导致过拟合）并记录。

七、参考代码框架

以下框架与 TRAE Builder 生成的代码作用相同，供对照与自查使用。模型类中标注了各改进点的插入位置：

```
import time
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei"] # 图中文字体
plt.rcParams["axes.unicode_minus"] = False
torch.manual_seed(42) # 全局随机种子，保证可复现

# ----- 1. 数据 -----
X, y = load_digits(return_X_y=True)
X = X.astype(np.float32) / 16.0 # 归一化到 [0, 1]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
X_tr = torch.tensor(X_tr)
y_tr = torch.tensor(y_tr, dtype=torch.long) # 必须 Long，否则报错
X_te = torch.tensor(X_te)
y_te = torch.tensor(y_te, dtype=torch.long)

# ----- 2. 模型 -----
class MLP(nn.Module):
    def __init__(self, hidden=(32,), act="sigmoid",
                 dropout=0.0, use_bn=False):
        super().__init__()
        act_layer = {"sigmoid": nn.Sigmoid, "tanh": nn.Tanh,
                     "relu": nn.ReLU, "gelu": nn.GELU}[act]
        layers, prev = [], 64
```

```

        for h in hidden:
            layers.append(nn.Linear(prev, h))
            if use_bn:                # 改进点②: BatchNorm
                layers.append(nn.BatchNorm1d(h))
            layers.append(act_layer())
            if dropout > 0:          # 改进点②: Dropout
                layers.append(nn.Dropout(dropout))
            prev = h
        layers.append(nn.Linear(prev, 10))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)

# ----- 3. 训练与评估 -----
def run(model, opt_name="sgd", lr=0.1, epochs=30, wd=0.0):
    if opt_name == "sgd":          # 改进点③: 优化器可切换
        opt = torch.optim.SGD(model.parameters(),
                               lr=lr, weight_decay=wd)
    else:
        opt = torch.optim.Adam(model.parameters(),
                                lr=lr, weight_decay=wd)
    loss_fn = nn.CrossEntropyLoss()
    hist = {"loss": [], "acc": []}
    t0 = time.time()
    for ep in range(epochs):
        model.train()
        opt.zero_grad()
        loss = loss_fn(model(X_tr), y_tr) # 前向 + 损失
        loss.backward()                  # 反向传播
        opt.step()                       # 参数更新
        model.eval()
        with torch.no_grad():
            acc = (model(X_te).argmax(1) == y_te) \
                  .float().mean().item()
        hist["loss"].append(loss.item()); hist["acc"].append(acc)
    return hist, time.time() - t0

if __name__ == "__main__":
    # 基线: 1x32 Sigmoid + SGD(0.1); 改参数即可完成各组对照实验
    model = MLP(hidden=(32,), act="sigmoid")
    hist, dt = run(model, opt_name="sgd", lr=0.1, epochs=30)
    print("最终测试准确率: %.2f%% 耗时: %.1fs"
          % (hist["acc"][-1] * 100, dt))

```

注意：做优化器对照（第3组）时只把 `opt_name` 改为 "adam"，其余不动；对照 Adam 通常搭配更小的学习率（如 0.01 或 0.001），可作为附加观察。

八、实验报告要求

实验报告以 Word / PDF 格式提交，命名格式为“学号_姓名_实验作业一.docx”。报告样板已单独提供（文件：实验作业一_实验报告样板.docx），样板中已排好封面、章节骨架、结果记录表与思考题作答区，请直接在样板中填写并插入曲线图后提交，结构与篇幅要求如下：

表 5 实验报告结构与篇幅建议

章节	内容要求	篇幅建议
1 实验概述	实验目的、环境（表 1 内容可简述）	0.5 页
2 基线实验	基线设置、运行结果、损失曲线截图	1 页
3 改进实验	逐组说明改动、动机、结果（引用表 4），附关键曲线	3-4 页
4 AI 协作记录	列出使用 TRAE 的具体场景：提示词原文、AI 输出摘要、你的采纳与修改	1 页
5 结果分析	最优组合、失败实验（学习率过大）分析、结论	1-2 页
6 思考题作答	回答第十一节思考题	0.5-1 页

曲线图要求：每张图必须有图号与图题（如“图 1 基线模型损失曲线”），坐标轴有标签，能被独立读懂。

九、评分标准

表 6 实验评分细则（满分 100 分）

评分项	分值	评分要点
实验完成度	30	完成基线 + 不少于 6 组对照实验，数据真实、记录完整
方法正确性	20	控制变量执行到位；随机种子固定；评估方式正确
结果与分析	25	能结合曲线与数据解释每组改进有效或无效的原因；失败实验分析到位
AI 协作质量	15	TRAE 使用场景清晰；对 AI 生成代码有理解、有修改、有判断，不是照搬
报告规范性	10	结构完整、图表编号规范、代码关键处有注释、行文通顺

诚信要求：允许并鼓励使用 TRAE 辅助编码，但表格与曲线中的每一个实验数字必须来自你自己的真实运行；发现编造数据者该项计 0 分。

十、常见问题与提示

Q1: CPU 训练会不会太慢？

本实验数据集只有 1797 个样本、30 个 epoch，CPU 上单组实验通常不超过 1 秒，无需 GPU。

Q2: 运行时报错 expected scalar type Long but found Int？

Windows 上 load_digits 的标签是 int32，而 CrossEntropyLoss 要求 Long 类型。转换方法：y_tr = torch.tensor(y_tr, dtype=torch.long)。

Q3: 画的图中中文显示成方框?

matplotlib 默认字体不含中文。画图前执行: `plt.rcParams["font.sans-serif"] = ["Microsoft YaHei"]` (Windows); macOS 可改为 `["Arial Unicode MS"]`, 并设置 `plt.rcParams["axes.unicode_minus"] = False`。

Q4: TRAE 生成的代码报错怎么办?

把完整报错信息粘贴到 Chat 面板, 让 AI 解释原因并给出修复; 也可以直接使用报错行旁的“修复”按钮。修复后务必看懂错误原因再继续。

Q5: 准确率每次运行略有不同?

若未固定随机种子 (`torch.manual_seed` 与 `train_test_split` 的 `random_state`), 数据划分与参数初始化都会变化。报告中的最优组合请固定种子跑 3 次取平均。

Q6: 加了很多改进, 准确率反而下降?

本实验中“加深到 2 层 $\times$ 128 (Sigmoid)”准确率大幅下降是正常现象: 深层 Sigmoid 网络梯度消失更严重, 且小数据集上容量过大。这本身就是有价值的实验结论, 如实记录并分析即可。

Q7: 可以用 MNIST 吗?

可以选做。MNIST 需联网下载 (`torchvision.datasets.MNIST`), 训练时间明显更长, 仅作为加分扩展。

十一、思考题 (在报告样板中作答)

1. 实验 1 中把 Sigmoid 换成 ReLU 后, 收敛速度发生了什么变化? 请结合 Sigmoid 导数最大值仅为 0.25 (梯度消失) 的数学性质解释原因。
2. 实验 4 中把学习率从 0.1 改为 0.01 时, SGD 与 Adam 受到的影响是否相同? 结合 Adam 自适应学习率的机制说明理由。
3. BatchNorm 为什么能让模型“容忍”更大的学习率? 结合内部协变量偏移 (Internal Covariate Shift) 的概念说明。
4. Dropout 在训练时丢弃神经元、在推理时却不丢弃, 推理时输出规模如何保持一致? 查阅资料说明 PyTorch 的处理方式。
5. 在本实验的小数据集上, 为什么“更大更深”的网络并不总是更好? 请用偏差—方差权衡 (或过拟合) 的语言给出解释。

6. 回顾你与 TRAE 的协作过程：AI 在哪些环节最有价值？哪些决策必须由人来做？用 200 字左右谈谈你对“人机协作做实验”的理解。

—— 实验作业一 完 ——